

SE3062 : Intelligent Systems

Practical 07 – Logical Reasoning & Inference

Faculty of Computing · Year 3 · Semester 1 · 2026 ·

Student ID: IT24101848

Student name: Perera H.L.P.S.

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

A new file logic_engine.py holds the KnowledgeBase class. Facts are stored in a set (unique strings) and rules are stored in a list of tuples ([premises], conclusion), i.e. Horn clauses. The same file also contains forward_chain() from Part 2.

```
# logic_engine.py
# Practical 7 - Knowledge Base and Forward Chaining inference engine



class KnowledgeBase:
    """Stores facts and Horn-clause rules, and deduces new facts by forward chaining."""

    def __init__(self):
        self.facts = set()   # unique string facts, e.g. "TargetVisible"
        self.rules = []      # tuples: ([premises], conclusion)

    def tell_fact(self, fact_string):
        self.facts.add(fact_string)

    def tell_rule(self, premise_list, conclusion_string):
        self.rules.append((list(premise_list), conclusion_string))

    def clear_facts(self):
        self.facts.clear()

    def forward_chain(self):
        new_facts_added = True

        while new_facts_added:
            new_facts_added = False

            for premises, conclusion in self.rules:
                if conclusion not in self.facts:
                    # Modus Ponens check
                    if all(p in self.facts for p in premises):
                        self.facts.add(conclusion)
                        new_facts_added = True
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

forward_chain() is the method at the bottom of the class above. An outer while loop repeats full passes over the rules. In every pass, each rule whose conclusion is not yet known is tested with all(); if every premise is a known fact, the conclusion is added and new_facts_added is set to True. The loop stops only when a complete pass adds nothing (a fixed point), so chained deductions (Rule 1 feeding Rule 2) are always found regardless of rule order.

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

- Added "from logic_engine import KnowledgeBase" to agent.py.
- Created self.kb = KnowledgeBase() in SearchAgent.__init__.
- Rule 1: TargetVisible ∧ HasDust ⇒ SafeToEngage.
- Rule 2: SafeToEngage ∧ BloodseekerMissing ⇒ Retreat.

Step 3.2: Validating Feasibility in A*

astar_search() gained a tile_facts=None parameter (None is treated as {}). In the neighbour loop, after the physical wall/bounds check done by _neighbors(), the KB is cleared, loaded with that tile's facts, and forward chained. If "Retreat" is in kb.facts the tile is skipped and never reaches the open list.

```
# agent.py - imports
from logic_engine import KnowledgeBase

# SearchAgent.__init__
self.kb = KnowledgeBase()
self.kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')      # Rule 1
self.kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat') # Rule 2

# astar_search(..., heuristic_type='manhattan', tile_facts=None)
if tile_facts is None:
    tile_facts = {}
...
for new_pos, action in self._neighbors(current_pos, wall_positions, grid_size):
    if new_pos not in reached_states:
        # Feasibility check: ask the Knowledge Base before expanding
        self.kb.clear_facts()
        for fact in tile_facts.get(new_pos, []):
            self.kb.tell_fact(fact)
        self.kb.forward_chain()
        if 'Retreat' in self.kb.facts:
            continue          # Infeasible: reachable but unsafe
        reached_states.add(new_pos)
        ...push to open list...
```

Step 3.3: Run It in the Grid

sense_and_act() defines the sample facts and passes them to A*:

```
# SearchAgent.sense_and_act()
if self.active_algo == 'AStar':
    tile_facts = {
        (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissing'],  # deduces Retreat: blocked
        (0, 1): ['TargetVisible', 'HasDust'],                        # SafeToEngage only: allowed
    }
    self.plan = self.astar_search(start, goal, percept['walls'], percept['grid_size'],
                                  tile_facts=tile_facts) or []
else:
    self.plan = search(start, goal, percept['walls'], percept['grid_size']) or []
```

Verification on an empty 5×5 grid, start (0,0), goal (2,0):

- Without tile_facts: path = [Right, Right] (goes straight through (1,0)).
- With tile_facts: path = [Up, Right, Right, Down] – (1,0) is blocked because Retreat is deduced, while (0,1) is still allowed because only SafeToEngage is deduced. The agent never steps on (1,0).

To see it in the GUI, run visual_grid_game.py and select the A* algorithm (the facts are applied when active_algo is "AStar").

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

test_logic.py contains the two supplied test cases (Safe Engagement; Unsafe Engagement with BloodseekerMissing). Running "python test_logic.py" prints:

```
✅ All Logic Engine Test Cases Passed!
```

The existing test_suite.py (unittest) also still passes (4 tests, OK).

Test case	Result
1: TargetVisible, HasDust	SafeToEngage deduced; Retreat NOT deduced – pass
2: + BloodseekerMissing	Retreat deduced (chain: rule 1 then rule 2) – pass

Part 5: Theory-to-Implementation Mapping

1. Reachability vs. Feasibility (Apply)

Reachability is a physical question: can the agent legally move onto the tile? In my code it is answered by _neighbors(), which only yields positions that are inside the grid and not in the walls set. This is a fixed property of the map. Feasibility is a logical question: even if the tile is reachable, is it safe to enter given what the agent currently knows? This is answered afterwards, inside the neighbour loop of astar_search(), by consulting the Knowledge Base.

The KB enforces feasibility as follows: for each candidate tile, kb.clear_facts() removes stale knowledge, the tile's percepts (tile_facts[tile]) are told to the KB as facts, and forward_chain() applies the Horn-clause rules to deduce new facts. If "Retreat" is derived, the tile is infeasible and the code uses continue, so it is never added to the open list or the reached set and A* can never route through it – even though it is physically reachable. Reachable tiles are therefore a superset of feasible tiles; the KB is a logical filter on top of the physical one.

2. Declarative vs. Procedural Paradigms (Analyze)

With a KnowledgeBase the safety knowledge is data (a list of rules), and a general-purpose inference procedure (forward_chain) is kept separate from it. The agent code only says "tell the facts, run inference, ask whether Retreat holds". Hard-coding "if 'TargetVisible' in percepts and 'HasDust' in percepts" inside the movement code mixes what is true about the world with how the agent acts, so every new rule needs new nested conditions.

Chained rules are the main problem for the procedural style: Retreat depends on SafeToEngage, which itself depends on two other facts, so the programmer must manually order and nest the checks and re-edit them whenever a rule changes. With 50 new rules, the if-else version grows combinatorially, becomes hard to read, test and debug, and is easy to get in the wrong order. In the declarative version we simply add 50 tell_rule() calls; the movement code and forward_chain() are unchanged, rule order does not matter because the loop runs to a fixed point, the rules can be loaded from a file or edited by a non-programmer, and each rule can be unit-tested in isolation. The cost is a little extra run-time (repeated passes over the rules), which is trivial here.

3. Modus Ponens & Horn Clauses (Understand)

A Horn clause has at most one positive conclusion: (P1 ∧ P2 ∧ … ∧ Pn) ⇒ Q. Modus Ponens says: if P1…Pn are all known to be true and the rule (P1 ∧ … ∧ Pn) ⇒ Q holds, then Q must be true. In

forward_chain(), the check all(p in self.facts for p in premises) is exactly the test that every premise is established; when it succeeds, adding the conclusion to self.facts is the application of Modus Ponens – a sound inference, because the conclusion is only added when it logically follows.

The while loop repeats this for all rules until a full pass adds no new fact. This is data-driven forward chaining: newly derived facts (e.g. SafeToEngage) become premises for other rules (e.g. Retreat) in later iterations, so repeated Modus Ponens yields every fact entailed by the Horn-clause KB, and the loop terminates because the set of possible facts is finite and each pass either adds a fact or stops.